

lasso_psm_analysis

nakada shunichi

1. データ読み込み

```
# =====  
# 0. 初期設定・パッケージ読み込み  
# =====  
library(tinytex)  
library(here)
```

here() starts at /Users/snakada/Library/CloudStorage/GoogleDrive-snakada@g.ecc.u-tokyo.ac.jp/マイドライブ/Peru_work

```
library(haven)  
library(glmnet)
```

Warning: package 'glmnet' was built under R version 4.5.1

Loading required package: Matrix

Loaded glmnet 4.1-10

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
# library(fastDummies)
library(nnet)    # 多項ロジスティック回帰
library(survey)  # 重み付き統計解析
```

Loading required package: grid

Loading required package: survival

Attaching package: 'survey'

The following object is masked from 'package:graphics':

dotchart

```
library(ggplot2)
library(did)     # Staggered DID用（必要に応じて）
library(sjlabelled) # ラベル付きデータ処理用
```

Attaching package: 'sjlabelled'

The following object is masked from 'package:ggplot2':

as_label

The following object is masked from 'package:dplyr':

as_label

The following objects are masked from 'package:haven':

as_factor, read_sas, read_spss, read_stata, write_sas, zap_labels

```
# オブジェクトクリア
rm(list = ls(all = TRUE))

# 自作関数読み込み
source(here("myTools.R"))
```

Attaching package: 'psych'

The following object is masked from 'package:did':

sim

The following objects are masked from 'package:ggplot2':

%+%, alpha

```
# =====
# 1. データ読み込み
# =====
path_data_file <- normalizePath(
  here("../output", "alldata", "merged_all_years.dta")
)
df_org <- read_dta(path_data_file)

# 変数ラベル取得
var_labels <- df_org %>% get_label %>% as_tibble() %>% rename(labels = value)
var_labels$varname <- names(df_org)
```

2. LASSO用データ前処理（介入前データのみ）

```
# =====
# 2. LASSO用データ前処理（介入前データのみ）
# =====

# 除外変数定義
drop_cols <- c(
  "caseid", "cluster_id", "strata_id", "state_id", "state",
  "treated_status", "time_period", "group_summary", "nt_ch_whz", "age",
```

```

"v001", "v002", "v003", "v005", "v008", "v013", "v022", "v024", "v025", "v026",
"b3", "b9", "bidx", "age_months", "b4", "v106", "v190", "hv024", "hv103",
"hv270", "hc1", "hc27", "sampling_weight", "treated_ever", "treated",
"treatment_cohort", "post_treatment", "dm_refrigerator", "nt_bbyfood",
"rc_tobc_other", "ph_wtr_trt_solar", "ms_sex_never", "ms_mar_union",
"ms_mar_never", "nt_ch_micro_vaf", "nt_solids", "nt_ch_sev_stunt",
"nt_ch_stunt", "nt_ch_mean_haz", "nt_ch_haz", "nt_ch_sev_wast", "nt_ch_wast",
"nt_ch_ovwt_ht", "nt_ch_mean_whz", "nt_ch_whz", "nt_ch_sev_underwt",
"nt_ch_underwt", "nt_ch_ovwt_age"
)

# ID・管理用変数
grouping_cols <- c("caseid", "cluster_id", "strata_id", "state_id", "state",
  "treated_status", "time_period", "group_summary",
  "treatment_start", "treated_ever", "treated",
  "treatment_cohort", "post_treatment", "sampling_weight")

# パターンマッチによる変数除外
drop_cols2 <- grep("^ch_pneumo|^ch_rotav", names(df_org), value = TRUE)
all_drop_cols <- c(drop_cols, drop_cols2, "treatment_start")

# 1. 介入前データで絞りこみ
df_pre <- df_org[df_org$treated_status %in% c("never_treated", "not_yet_treated"), ]

# 3. 介入群グループ化
df_pre <- df_pre %>%
  mutate(
    treatment_group = case_when(
      treatment_start == "2009" ~ 1, # 早期介入
      treatment_start == "2011" ~ 2, # 後期介入
      TRUE ~ 0 # コントロール
    ),
    treatment_group = as.factor(treatment_group)
  )

cat("介入前データの群別サンプル数:\n")

```

介入前データの群別サンプル数:

```
print(table(df_pre$treatment_group))
```

```
0 1 2
5156 292 526
```

```
# 4. NAの多い列を除去（10%以上NA）
initial_cols <- ncol(df_pre)
keep_cols <- names(df_pre)[sapply(df_pre, function(x) sum(is.na(x)) <= nrow(df_pre) * 0.1)]
df_pre <- df_pre[, keep_cols]
cat("NA値の多い列を", initial_cols - ncol(df_pre), "個削除しました\n")
```

NA値の多い列を 100 個削除しました

```
# 5. 共変量名の再定義（NA除去後）
all_drop_cols <- intersect(all_drop_cols, names(df_pre))
grouping_cols <- intersect(grouping_cols, names(df_pre))
covariate_names <- setdiff(names(df_pre), c(all_drop_cols, grouping_cols, "treatment_group"))

# 6. 共変量のNAがある行を除去
complete_idx <- complete.cases(df_pre[, covariate_names])
initial_rows <- nrow(df_pre)
df_pre <- df_pre[complete_idx, ]
cat("NA値を含む行を", initial_rows - nrow(df_pre), "行削除しました\n")
```

NA値を含む行を 783 行削除しました

```
cat("最終的な介入前データサイズ:", nrow(df_pre), "行 ×", ncol(df_pre), "列\n\n")
```

最終的な介入前データサイズ: 5191 行 × 239 列

3. LASSO変数選択（介入前データ）

```
# =====
# 3. LASSO変数選択（介入前データ）
# =====

# 説明変数マトリックス作成
X_matrix <- model.matrix(~ . - 1, data = df_pre[, covariate_names, drop=FALSE])
X_scaled <- scale(X_matrix)
Y <- df_pre$treatment_group

cat("説明変数の数:", ncol(X_scaled), "\n")
```

説明変数の数: 169

```
cat("目的変数の水準:", levels(Y), "\n\n")
```

目的変数の水準: 0 1 2

```
# 多項分類LASSO実行  
cat("多項分類LASSO回帰を実行中...\n")
```

多項分類LASSO回帰を実行中...

```
set.seed(123) # 再現性のため  
cv_model <- cv.glmnet(X_scaled, Y, alpha = 1, family = "multinomial", nfolds = 10)  
  
cat("最適  $\lambda$  (lambda.min):", cv_model$lambda.min, "\n")
```

最適 λ (lambda.min): 0.0006454161

```
cat("1se  $\lambda$  (lambda.1se):", cv_model$lambda.1se, "\n\n")
```

1se λ (lambda.1se): 0.003138401

```
# 選択された変数の抽出  
coef_min <- coef(cv_model, s = "lambda.min")  
selected_vars <- c()  
for(i in 1:length(coef_min)) {  
  coef_matrix <- as.matrix(coef_min[[i]])  
  class_vars <- rownames(coef_matrix)[coef_matrix[,1] != 0 & rownames(coef_matrix) != "(Intercept)"]  
  selected_vars <- union(selected_vars, class_vars)  
}  
  
cat("LASSO選択変数数:", length(selected_vars), "\n")
```

LASSO選択変数数: 150

```

if(length(selected_vars) > 0) {
  cat("選択された変数:\n")
  # for(i in 1:length(selected_vars)) {
  #   cat(i, ":", selected_vars[i], "\n")
  # }
  # 変数の重要度確認
  coef_importance <- abs(as.matrix(coef(cv_model, s = "lambda.min"))[[1]])
  top_vars <- head(sort(coef_importance[,1], decreasing = TRUE), 20)
  coef_importance <- coef_importance %>%
    as_tibble(rownames = "labels") %>%
    filter(labels %in% selected_vars) %>%
    left_join(var_labels, by = join_by("labels" == "varname"))
  print(coef_importance)
} else {
  stop("変数が選択されませんでした。λの値を調整してください。")
}

```

選択された変数:

A tibble: 150 x 3

labels	lambda.min	labels.y
<chr>	<dbl>	<chr>
1 rh_del_pltype	0	Live births by type of health facility
2 rh_del_pv	0.0255	Person providing assistance during delivery
3 rh_del_ces	0	Live births delivered by Caesarean
4 year2005	0	<NA>
5 year2008	0	<NA>
6 year2009	0	<NA>
7 year2010	0	<NA>
8 year2012	0.152	<NA>
9 ph_sani_type	0	type of toilet facility
10 ph_sani_improve	0.700	Improved sanitation

i 140 more rows

4. 多項GLM-PSM（介入前データで学習）

```

# =====
# 4. 多項GLM-PSM（介入前データで学習）
# =====
if(length(selected_vars) > 0) {

```

```

# LASSO と同じ model.matrix 処理を適用
psm_matrix <- model.matrix(~ . - 1, data = df_pre[, covariate_names, drop=FALSE])

# 選択された変数のみ抽出
psm_covariates <- as.data.frame(psm_matrix[, selected_vars, drop=FALSE])

# 目的変数と管理変数を追加
psm_data <- cbind(
  treatment_group = df_pre$treatment_group,
  psm_covariates,
  df_pre[, intersect(grouping_cols, names(df_pre))]
)

# 多項ロジスティック回帰で傾向スコア推定
psm_formula <- as.formula(paste("treatment_group ~", paste(selected_vars, collapse = " + ")))

cat("\n多項ロジスティック回帰で傾向スコア推定中...\n")
psm_model <- multinom(psm_formula, data = psm_data, trace = FALSE)

# 傾向スコア（各群の確率）を計算
pscore_matrix <- predict(psm_model, type = "probs")

# デバッグ情報
cat("pscore_matrix の次元:", dim(pscore_matrix), "\n")
cat("treatment_group の水準:", levels(psm_data$treatment_group), "\n")
cat("treatment_group の値:", table(psm_data$treatment_group), "\n")

# データフレーム形式に変換
if(is.vector(pscore_matrix)) {
  # 2群の場合の処理
  pscore_df <- data.frame(
    group_0 = 1 - pscore_matrix,
    group_1 = pscore_matrix
  )
  names(pscore_df) <- paste0("prob_", levels(psm_data$treatment_group))
} else {
  # 3群以上の場合
  pscore_df <- as.data.frame(pscore_matrix)
  # 列名を明示的に設定
  names(pscore_df) <- paste0("prob_", levels(psm_data$treatment_group))
}

```

```

# IPTW重み計算（より安全な方法）
psm_data$iptw_weight <- numeric(nrow(psm_data))

for(i in 1:nrow(psm_data)) {
  group_level <- as.character(psm_data$treatment_group[i])
  prob_col <- paste0("prob_", group_level)

  if(prob_col %in% names(pscore_df)) {
    psm_data$iptw_weight[i] <- 1 / pscore_df[i, prob_col]
  } else {
    cat("警告: 群", group_level, "の確率列が見つかりません\n")
    psm_data$iptw_weight[i] <- 1 # デフォルト重み
  }
}

cat("PSM完了。重みの要約統計:\n")
print(summary(psm_data$iptw_weight))

# 極端な重みの確認
extreme_weights <- psm_data$iptw_weight > quantile(psm_data$iptw_weight, 0.99, na.rm = TRUE)
if(sum(extreme_weights, na.rm = TRUE) > 0) {
  cat("警告: 極端に大きな重みが", sum(extreme_weights, na.rm = TRUE), "個あります\n")
}

} else {
  stop("選択された変数がありません。")
}

```

多項ロジスティック回帰で傾向スコア推定中...

pscore_matrix の次元: 5191 3

treatment_group の水準: 0 1 2

treatment_group の値: 4450 264 477

PSM完了。重みの要約統計:

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.000	1.001	1.041	1.822	1.238	93.632

警告: 極端に大きな重みが 52 個あります

5. 介入後+コントロールデータへの適用

```
# =====
# 5. 介入後+コントロールデータへの適用
# =====

# 介入後+コントロールデータ抽出
df_post <- df_org[df_org$treated_status %in% c("never_treated", "early_treated", "late_treated"), ]

# 介入群の再定義
df_post <- df_post %>%
  mutate(
    treatment_group = case_when(
      treated_status == "never_treated" ~ 0,
      treated_status == "early_treated" ~ 1,
      treated_status == "late_treated" ~ 2,
      TRUE ~ 0
    ),
    treatment_group = as.factor(treatment_group)
  )

cat("\n介入後データの群別サンプル数:\n")
```

介入後データの群別サンプル数:

```
print(table(df_post$treatment_group))
```

```
 0  1  2
5156 466 260
```

```
# 共変量を介入前と同じ形式に調整
missing_vars <- setdiff(selected_vars, names(df_post))
if(length(missing_vars) > 0) {
  cat("警告: 以下の変数が介入後データに存在しません:", paste(missing_vars, collapse = ", "), "\n")
  # 欠損変数を0で補完
  for(var in missing_vars) {
    df_post[[var]] <- 0
  }
}
```

警告: 以下の変数が介入後データに存在しません: year2012, year2005, year2008, year2009, year2010

```
# NAを含む行を除去
post_complete_idx <- complete.cases(df_post[, c(selected_vars, grouping_cols, "nt_ch_stunt")])
df_post <- df_post[post_complete_idx, ]

# 学習済みモデルで傾向スコア予測
cat("学習済みモデルで介入後データに傾向スコア適用中...\n")
```

学習済みモデルで介入後データに傾向スコア適用中...

```
# 介入後データも model.matrix で処理 (PSM学習時と同じ形式に)
post_matrix <- model.matrix(~ . - 1, data = df_post[, covariate_names, drop=FALSE])
post_covariates <- as.data.frame(post_matrix[, selected_vars, drop=FALSE])

# 予測用データフレーム作成
predict_data <- cbind(
  treatment_group = df_post$treatment_group,
  post_covariates
)

# 傾向スコア予測
post_pscore_matrix <- predict(psm_model, newdata = predict_data, type = "probs")

# デバッグ情報
cat("post_pscore_matrix の次元:", dim(post_pscore_matrix), "\n")
```

post_pscore_matrix の次元: 5069 3

```
cat("df_post の treatment_group:", table(df_post$treatment_group), "\n")
```

df_post の treatment_group: 4450 404 215

```
# データフレーム形式に変換
if(is.vector(post_pscore_matrix)) {
  # 2群の場合
  post_pscore_df <- data.frame(
    group_0 = 1 - post_pscore_matrix,
    group_1 = post_pscore_matrix
  )
}
```

```

names(post_pscore_df) <- paste0("prob_", levels(df_post$treatment_group))
} else {
  # 3群以上の場合
  post_pscore_df <- as.data.frame(post_pscore_matrix)
  names(post_pscore_df) <- paste0("prob_", levels(df_post$treatment_group))
}

# IPTW重み計算（安全な方法）
df_post$iptw_weight <- numeric(nrow(df_post))

for(i in 1:nrow(df_post)) {
  group_level <- as.character(df_post$treatment_group[i])
  prob_col <- paste0("prob_", group_level)

  if(prob_col %in% names(post_pscore_df)) {
    prob_value <- post_pscore_df[i, prob_col]
    # 極小確率値の処理（ゼロ除算防止）
    if(prob_value > 0.001) {
      df_post$iptw_weight[i] <- 1 / prob_value
    } else {
      df_post$iptw_weight[i] <- 1 / 0.001 # 最大重み制限
      if(i <= 10) cat("警告: 極小確率値を調整しました（行", i, "）\n")
    }
  } else {
    cat("警告: 群", group_level, "の確率列が見つかりません（行", i, "）\n")
    df_post$iptw_weight[i] <- 1
  }
}

cat("介入後データの重み計算完了\n")

```

介入後データの重み計算完了

```
print(summary(df_post$iptw_weight))
```

```

Min. 1st Qu. Median Mean 3rd Qu. Max.
1.000 1.001 1.039 82.173 1.248 1000.000

```

```

# サンプル重みとの組み合わせ（v005を1,000,000で割って正規化）
df_post$sampling_weight_norm <- df_post$v005 / 1000000

```

```
df_post$w_combined <- df_post$iptw_weight * df_post$sampling_weight_norm

# 極端な重みを調整（オプション）
weight_quantiles <- quantile(df_post$w_combined, c(0.01, 0.99), na.rm = TRUE)
df_post$w_combined_trimmed <- pmax(pmin(df_post$w_combined, weight_quantiles[2]), weight_quantiles[1])

cat("重み付きデータ準備完了。最終サンプル数:", nrow(df_post), "\n")
```

重み付きデータ準備完了。最終サンプル数: 5069

```
cat("重みの要約統計:\n")
```

重みの要約統計:

```
print(summary(df_post$w_combined_trimmed))
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.07055 0.38047 0.98013 49.13030 1.87526 900.59748
```

6. PSMのバランスチェック（クラスターサンプリングによる重み付けなし）

```
# =====
# PSMバランス確認（重み付けなし）
# =====
# =====
# 介入前と介入後のグループで変数選定が異なる（データ処理の途中で
# dropされる変数があるため、2通りの方法で検証）
# =====

# 安全なSMD計算関数
calculate_smd_robust <- function(data_matrix, group_vector, vars) {
  results <- data.frame(
    variable = character(0),
    smd = numeric(0),
    mean_control = numeric(0),
    mean_treated = numeric(0),
    stringsAsFactors = FALSE
```

```

)

for(i in 1:length(vars)) {
  var <- vars[i]

  # 変数が存在するかチェック
  if(!var %in% colnames(data_matrix)) {
    next # 存在しない変数はスキップ
  }

  # データの抽出 (try-catchで安全に)
  tryCatch({
    control_data <- data_matrix[group_vector == 0, var]
    treated_data <- data_matrix[group_vector == 1, var]

    # 数値型かチェック
    if(!is.numeric(control_data) || !is.numeric(treated_data)) {
      next
    }

    # 基本統計量の計算
    mean_c <- mean(control_data, na.rm = TRUE)
    mean_t <- mean(treated_data, na.rm = TRUE)
    var_c <- var(control_data, na.rm = TRUE)
    var_t <- var(treated_data, na.rm = TRUE)

    # NA値の処理
    if(is.na(mean_c) || is.na(mean_t) || is.na(var_c) || is.na(var_t)) {
      next
    }

    # SMD計算 (安全版)
    if(var_c == 0 && var_t == 0) {
      # 両群の分散が0の場合
      smd <- 0
    } else if(var_c == 0 || var_t == 0 || (var_c + var_t) == 0) {
      # いずれかの分散が0または合計が0の場合
      # 単純な平均差を使用 (絶対値)
      smd <- abs(mean_t - mean_c)
    } else {
      # 通常のSMD計算
      smd <- abs((mean_t - mean_c) / sqrt((var_t + var_c) / 2))
    }
  }, error = function(e) {
    next
  })
}

```

```

}

# 結果を追加
results <- rbind(results, data.frame(
  variable = var,
  smd = smd,
  mean_control = mean_c,
  mean_treated = mean_t,
  stringsAsFactors = FALSE
))

}, error = function(e) {
  # エラーが発生した変数は静かにスキップ
  cat("変数", var, "でエラー:", e$message, "\n")
})
}

return(results)
}

# 簡易バランス確認関数（カテゴリ変数対応）
calculate_balance_simple <- function(data, vars, group_var) {
  results <- data.frame(
    variable = character(0),
    smd = numeric(0),
    prop_control = numeric(0),
    prop_treated = numeric(0),
    stringsAsFactors = FALSE
  )

  # 存在する変数のみをフィルタリング
  existing_vars <- intersect(vars, names(data))

  for(var in existing_vars) {
    tryCatch({
      # データの抽出
      control_group <- data[data[[group_var]] == 0, ]
      treated_group <- data[data[[group_var]] == 1, ]

      # 変数の型を確認
      if(is.numeric(data[[var]])) {
        # 数値変数の場合

```

```

mean_c <- mean(control_group[[var]], na.rm = TRUE)
mean_t <- mean(treated_group[[var]], na.rm = TRUE)
var_c <- var(control_group[[var]], na.rm = TRUE)
var_t <- var(treated_group[[var]], na.rm = TRUE)

# SMD計算
if(is.na(var_c) || is.na(var_t) || (var_c + var_t) <= 0) {
  smd <- abs(mean_t - mean_c)
} else {
  smd <- abs((mean_t - mean_c) / sqrt((var_t + var_c) / 2))
}

results <- rbind(results, data.frame(
  variable = var,
  smd = smd,
  prop_control = mean_c,
  prop_treated = mean_t,
  stringsAsFactors = FALSE
))

} else {
  # カテゴリ変数の場合（割合で評価）
  # 最頻値または "1" の割合を計算
  if(all(data[[var]] %in% c(0, 1), na.rm = TRUE)) {
    # バイナリ変数の場合
    prop_c <- mean(control_group[[var]] == 1, na.rm = TRUE)
    prop_t <- mean(treated_group[[var]] == 1, na.rm = TRUE)
  } else {
    # その他のカテゴリ変数の場合（最頻値の割合）
    mode_val <- names(sort(table(data[[var]]), decreasing = TRUE))[1]
    prop_c <- mean(control_group[[var]] == mode_val, na.rm = TRUE)
    prop_t <- mean(treated_group[[var]] == mode_val, na.rm = TRUE)
  }

  # 割合のSMD（標準化差）
  pooled_var <- prop_c * (1 - prop_c) + prop_t * (1 - prop_t)
  if(pooled_var > 0) {
    smd <- abs((prop_t - prop_c) / sqrt(pooled_var / 2))
  } else {
    smd <- abs(prop_t - prop_c)
  }
}

```

```

    results <- rbind(results, data.frame(
      variable = var,
      smd = smd,
      prop_control = prop_c,
      prop_treated = prop_t,
      stringsAsFactors = FALSE
    ))
  }

  }, error = function(e) {
    # エラー時は静かにスキップ
  })
}

return(results)
}

# =====
# 実際のバランス確認実行（修正版）
# =====

cat("\n=== 修正版バランス確認開始 ===\n")

```

=== 修正版バランス確認開始 ===

```

# Method 1: model.matrixを使用（エラー処理強化）
cat("\n=== Method 1: model.matrix（エラー処理強化版） ===\n")

```

=== Method 1: model.matrix（エラー処理強化版） ===

```

tryCatch({
  # PSM学習時と同じ処理
  pre_matrix <- model.matrix(~ . - 1, data = df_pre[, covariate_names, drop=FALSE])
  post_matrix <- model.matrix(~ . - 1, data = df_post[, covariate_names, drop=FALSE])

  # 共通する変数のみを抽出
  common_vars <- intersect(selected_vars, intersect(colnames(pre_matrix), colnames(post_matrix)))
  cat("共通変数数:", length(common_vars), "/", length(selected_vars), "\n")

```

```

if(length(common_vars) > 0) {
  # バランス確認実行（安全版）
  balance_before_matrix <- calculate_smd_robust(
    pre_matrix,
    as.numeric(df_pre$treatment_group),
    common_vars
  )

  balance_after_matrix <- calculate_smd_robust(
    post_matrix,
    as.numeric(df_post$treatment_group),
    common_vars
  )

  cat("計算できた変数数 - Before:", nrow(balance_before_matrix),
      "/ After:", nrow(balance_after_matrix), "\n")

  # 結果のマージ
  if(nrow(balance_before_matrix) > 0 && nrow(balance_after_matrix) > 0) {
    balance_comparison_matrix <- merge(
      balance_before_matrix,
      balance_after_matrix,
      by = "variable",
      suffixes = c("_before", "_after")
    )

    balance_comparison_matrix$improvement <-
      balance_comparison_matrix$smd_before - balance_comparison_matrix$smd_after

    # 結果の要約
    cat("\n=== バランス改善結果 (Method 1) ===\n")
    cat("総変数数:", nrow(balance_comparison_matrix), "\n")
    cat("SMD < 0.1 (良好) - Before:",
        sum(balance_comparison_matrix$smd_before < 0.1, na.rm = TRUE),
        "/ After:",
        sum(balance_comparison_matrix$smd_after < 0.1, na.rm = TRUE), "\n")
    cat("SMD < 0.05 (優秀) - Before:",
        sum(balance_comparison_matrix$smd_before < 0.05, na.rm = TRUE),
        "/ After:",
        sum(balance_comparison_matrix$smd_after < 0.05, na.rm = TRUE), "\n")

    # 平均SMDの改善

```

```

cat("平均SMD - Before:",
    round(mean(balance_comparison_matrix$smd_before, na.rm = TRUE), 4),
    "/ After:",
    round(mean(balance_comparison_matrix$smd_after, na.rm = TRUE), 4), "\n")

# 最も改善された変数 top 5
if(nrow(balance_comparison_matrix) > 0) {
  top_improved <- head(
    balance_comparison_matrix[order(-balance_comparison_matrix$improvement), ],
    5
  )
  cat("\n最も改善された変数 Top 5:\n")
  print(top_improved[, c("variable", "smd_before", "smd_after", "improvement")])

  # 最もバランスが悪い変数 top 5
  worst_balance <- head(
    balance_comparison_matrix[order(-balance_comparison_matrix$smd_after), ],
    5
  )
  cat("\n最もバランスが悪い変数 Top 5 (After):\n")
  print(worst_balance[, c("variable", "smd_before", "smd_after", "improvement")])
}
}
}
}, error = function(e) {
  cat("Method 1でエラーが発生しました:", e$message, "\n")
})

```

共通変数数: 150 / 150

計算できた変数数 - Before: 0 / After: 0

```

# Method 2: 簡易版（カテゴリ変数対応）
cat("\n\n=== Method 2: 簡易版（全変数対応） ===\n")

```

=== Method 2: 簡易版（全変数対応） ===

```

tryCatch({
  balance_before_simple <- calculate_balance_simple(df_pre, selected_vars, "treatment_group")

```

```

balance_after_simple <- calculate_balance_simple(df_post, selected_vars, "treatment_group")

cat("計算できた変数数 - Before:", nrow(balance_before_simple),
    "/ After:", nrow(balance_after_simple), "\n")

if(nrow(balance_before_simple) > 0 && nrow(balance_after_simple) > 0) {
  balance_comparison_simple <- merge(
    balance_before_simple,
    balance_after_simple,
    by = "variable",
    suffixes = c("_before", "_after")
  )

  balance_comparison_simple$improvement <-
    balance_comparison_simple$smd_before - balance_comparison_simple$smd_after

  cat("\n=== バランス改善結果 (Method 2) ===\n")
  cat("総変数数:", nrow(balance_comparison_simple), "\n")
  cat("SMD < 0.1 (良好) - Before:",
    sum(balance_comparison_simple$smd_before < 0.1, na.rm = TRUE),
    "/ After:",
    sum(balance_comparison_simple$smd_after < 0.1, na.rm = TRUE), "\n")
  cat("SMD < 0.05 (優秀) - Before:",
    sum(balance_comparison_simple$smd_before < 0.05, na.rm = TRUE),
    "/ After:",
    sum(balance_comparison_simple$smd_after < 0.05, na.rm = TRUE), "\n")

  # 平均SMDの改善
  cat("平均SMD - Before:",
    round(mean(balance_comparison_simple$smd_before, na.rm = TRUE), 4),
    "/ After:",
    round(mean(balance_comparison_simple$smd_after, na.rm = TRUE), 4), "\n")
}
}, error = function(e) {
  cat("Method 2でエラーが発生しました:", e$message, "\n")
})

```

計算できた変数数 - Before: 145 / After: 150

=== バランス改善結果 (Method 2) ===

総変数数: 145

SMD < 0.1 (良好) - Before: 51 / After: 60

SMD < 0.05 (優秀) - Before: 18 / After: 31
平均SMD - Before: 0.2313 / After: 0.2203

```
# =====  
# 結果の可視化（エラー処理付き）  
# =====  
  
tryCatch({  
  if(exists("balance_comparison_matrix") && nrow(balance_comparison_matrix) > 0) {  
    library(ggplot2)  
  
    # SMD改善のヒストグラム  
    p_hist <- ggplot(balance_comparison_matrix, aes(x = improvement)) +  
      geom_histogram(bins = 20, alpha = 0.7, fill = "skyblue") +  
      geom_vline(xintercept = 0, color = "red", linetype = "dashed") +  
      labs(  
        title = "Distribution of SMD Improvement (After - Before)",  
        x = "SMD Improvement",  
        y = "Number of Variables",  
        caption = "Positive values indicate improvement"  
      ) +  
      theme_minimal()  
  
    print(p_hist)  
  
    # Before vs After SMDの散布図  
    p_scatter <- ggplot(balance_comparison_matrix, aes(x = smd_before, y = smd_after)) +  
      geom_point(alpha = 0.6, size = 2) +  
      geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed") +  
      geom_hline(yintercept = 0.1, color = "blue", linetype = "dotted") +  
      geom_vline(xintercept = 0.1, color = "blue", linetype = "dotted") +  
      labs(  
        title = "SMD: Before vs After PSM",  
        x = "SMD Before PSM",  
        y = "SMD After PSM",  
        caption = "Points below diagonal line show improvement\nBlue lines: SMD = 0.1 threshold"  
      ) +  
      theme_minimal()  
  
    print(p_scatter)  
  }  
}, error = function(e) {
```

```
cat("可視化でエラーが発生しました:", e$message, "\n")
})

cat("\n=== バランス確認完了（修正版） ===\n")
```

=== バランス確認完了（修正版） ===

7. 重み付き多群比較分析

```
# =====
# 7. 重み付き多群比較分析
# =====

# Survey design作成
design_post <- svydesign(
  ids = ~v001, # DHS Primary Sampling Unit
  strata = ~v022, # DHS strata
  weights = ~w_combined_trimmed,
  data = df_post,
  nest = TRUE
)

# 孤立PSUの処理
options(survey.lonely.psu = "adjust")

cat("\n=== 重み付き群別平均値 ===\n")
```

=== 重み付き群別平均値 ===

```
group_means <- svyby(~nt_ch_stunt, ~treatment_group, design = design_post, svymean, na.rm = TRUE)
print(group_means)
```

	treatment_group	nt_ch_stunt	se
0	0	0.2387344	0.01095498
1	1	0.4532513	0.03183511
2	2	0.3322788	0.03764158

```
# 2群間t検定
cat("\n=== 2群間t検定 ===\n")
```

=== 2群間t検定 ===

```
# コントロール vs 早期介入
cat("コントロール群 vs 早期介入群:\n")
```

コントロール群 vs 早期介入群:

```
design_01 <- subset(design_post, treatment_group %in% c(0, 1))
design_01 <- update(design_01, treatment_group = droplevels(treatment_group))
if(nrow(design_01$variables) > 0 && length(unique(design_01$variables$treatment_group)) == 2) {
  result_01 <- svytest(nt_ch_stunt ~ treatment_group, design = design_01)
  print(result_01)
} else {
  cat("十分なサンプルがありません\n")
}
```

Design-based t-test

```
data: nt_ch_stunt ~ treatment_group
t = 6.3717, df = 2322, p-value = 2.247e-10
alternative hypothesis: true difference in mean is not equal to 0
95 percent confidence interval:
 0.1484958 0.2805380
sample estimates:
difference in mean
      0.2145169
```

```
# コントロール vs 後期介入
cat("\nコントロール群 vs 後期介入群:\n")
```

コントロール群 vs 後期介入群:

```

design_02 <- subset(design_post, treatment_group %in% c(0, 2))
design_02 <- update(design_02, treatment_group = droplevels(treatment_group))
if(nrow(design_02$variables) > 0 && length(unique(design_02$variables$treatment_group)) == 2) {
  result_02 <- svytest(nt_ch_stunt ~ treatment_group, design = design_02)
  print(result_02)
} else {
  cat("十分なサンプルがありません\n")
}

```

Design-based t-test

```

data: nt_ch_stunt ~ treatment_group
t = 2.3861, df = 2240, p-value = 0.01711
alternative hypothesis: true difference in mean is not equal to 0
95 percent confidence interval:
 0.01666579 0.17042307
sample estimates:
difference in mean
 0.09354443

```

```

# 早期介入 vs 後期介入
cat("\n早期介入群 vs 後期介入群:\n")

```

早期介入群 vs 後期介入群:

```

design_12 <- subset(design_post, treatment_group %in% c(1, 2))
design_12 <- update(design_12, treatment_group = droplevels(treatment_group))
if(nrow(design_12$variables) > 0 && length(unique(design_12$variables$treatment_group)) == 2) {
  result_12 <- svytest(nt_ch_stunt ~ treatment_group, design = design_12)
  print(result_12)
} else {
  cat("十分なサンプルがありません\n")
}

```

Design-based t-test

```

data: nt_ch_stunt ~ treatment_group

```

t = -2.4539, df = 359, p-value = 0.01461
 alternative hypothesis: true difference in mean is not equal to 0
 95 percent confidence interval:
 -0.21792303 -0.02402193
 sample estimates:
 difference in mean
 -0.1209725

```
# 3群ANOVA
cat("\n=== 3群ANOVA ===\n")
```

=== 3群ANOVA ===

```
anova_model <- svyglm(nt_ch_stunt ~ factor(treatment_group), design = design_post)
print(summary(anova_model))
```

Call:

```
svyglm(formula = nt_ch_stunt ~ factor(treatment_group), design = design_post)
```

Survey design:

```
svydesign(ids = ~v001, strata = ~v022, weights = ~w_combined_trimmed,
  data = df_post, nest = TRUE)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.23873	0.01095	21.792	< 2e-16 ***
factor(treatment_group)1	0.21452	0.03367	6.372	2.22e-10 ***
factor(treatment_group)2	0.09354	0.03920	2.386	0.0171 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.2371007)

Number of Fisher Scoring iterations: 2

```
print(anova(anova_model))
```

NULL

8. Staggered DID

```
# =====  
# 8. Staggered DID (オプション)  
# =====  
  
cat("\n=== Staggered DID分析 ===\n")
```

=== Staggered DID分析 ===

```
# DID用データ準備  
did_data <- df_org %>%  
  mutate(  
    G = case_when(  
      treated_status == "never_treated" ~ 0,  
      treatment_start == "2009" ~ 2009,  
      treatment_start == "2011" ~ 2011,  
      TRUE ~ 0  
    ),  
    T = as.integer(year),  
    D = as.integer(treated_status != "never_treated" & post_treatment == 1),  
    unit_id = as.integer(as.factor(paste(caseid, cluster_id, sep = "_")))  
  ) %>%  
  filter(!is.na(nt_ch_stunt), !is.na(G), !is.na(T)) %>%  
  arrange(unit_id, T)  
  
# IPTWウェイトをDIDデータにマージ (caseid等で結合)  
if("w_combined_trimmed" %in% names(df_post)) {  
  did_weights <- df_post %>%  
    select(caseid, year, w_combined_trimmed) %>%  
    mutate(T = as.integer(year))  
  
  did_data <- did_data %>%  
    left_join(did_weights, by = c("caseid", "T")) %>%  
    mutate(weights = ifelse(is.na(w_combined_trimmed), 1, w_combined_trimmed))  
} else {  
  did_data$weights <- 1  
}  
  
# DID実行 (did パッケージ使用)
```

```

if(requireNamespace("did", quietly = TRUE)) {
  tryCatch({
    did_result <- did::att_gt(
      yname = "nt_ch_stunt",
      tname = "T",
      idname = "unit_id",
      gname = "G",
      data = did_data,
      weightsname = "weights",
      panel = FALSE,
      control_group = "nevertreated"
    )

    # 結果要約
    did_summary <- did::aggte(did_result, type = "group")
    print(did_summary)

    # 動的効果のプロット
    did_dynamic <- did::aggte(did_result, type = "dynamic")
    print(did_dynamic)

  }, error = function(e) {
    cat("DID分析でエラーが発生しました:", e$message, "\n")
    cat("データ構造やサンプル数を確認してください\n")
  })
} else {
  cat("didパッケージがインストールされていません。install.packages('did')でインストールしてください\n")
}

```

Call:

```
did::aggte(MP = did_result, type = "group")
```

Reference: Callaway, Brantly and Pedro H.C. Sant'Anna. "Difference-in-Differences with Multiple Time Periods." Journal of Econometrics 230, 2021. <<https://doi.org/10.1016/j.jeconom.2020.12.001>>, <<https://arxiv.org/abs/1803.09015>>

Overall summary of ATT's based on group/cohort aggregation:

ATT	Std. Error	[95% Conf. Int.]
0.0639	0.0427	-0.0197 0.1476

Group Effects:

Group Estimate Std. Error [95% Simult. Conf. Band]

2009 0.0491 0.0557 -0.0826 0.1809

2011 0.0888 0.0583 -0.0490 0.2267

Signif. codes: `*' confidence band does not cover 0

Control Group: Never Treated, Anticipation Periods: 0

Estimation Method: Doubly Robust

Call:

did::aggte(MP = did_result, type = "dynamic")

Reference: Callaway, Brantly and Pedro H.C. Sant'Anna. "Difference-in-Differences with Multiple Time Periods." Journal of 230, 2021. <<https://doi.org/10.1016/j.jeconom.2020.12.001>>, <<https://arxiv.org/abs/1803.09015>>

Overall summary of ATT's based on event-study/dynamic aggregation:

ATT Std. Error [95% Conf. Int.]

0.0622 0.0458 -0.0275 0.1519

Dynamic Effects:

Event time Estimate Std. Error [95% Simult. Conf. Band]

-3 0.0000 0.0653 -0.1755 0.1755

-2 -0.0803 0.0666 -0.2594 0.0989

-1 -0.0188 0.0477 -0.1471 0.1095

0 0.0367 0.0508 -0.0999 0.1732

1 0.0532 0.0549 -0.0943 0.2007

2 0.2148 0.0878 -0.0214 0.4509

3 -0.0559 0.0761 -0.2606 0.1488

Signif. codes: `*' confidence band does not cover 0

Control Group: Never Treated, Anticipation Periods: 0

Estimation Method: Doubly Robust

9. 結果の保存・可視化

```
# =====
# 9. 結果の保存・可視化 \# \#
```

```
# =====
writeLines(c( "=== 分析完了 ===", "主要結果:", paste("1.
LASSO選択変数数:", length(selected_vars)),
paste("2.最終分析サンプル数:", nrow(df_post)), "3. 群別サンプル数:" ))
```

=== 分析完了 ===

主要結果:

1.

LASSO選択変数数: 150

2.最終分析サンプル数: 5069

3. 群別サンプル数:

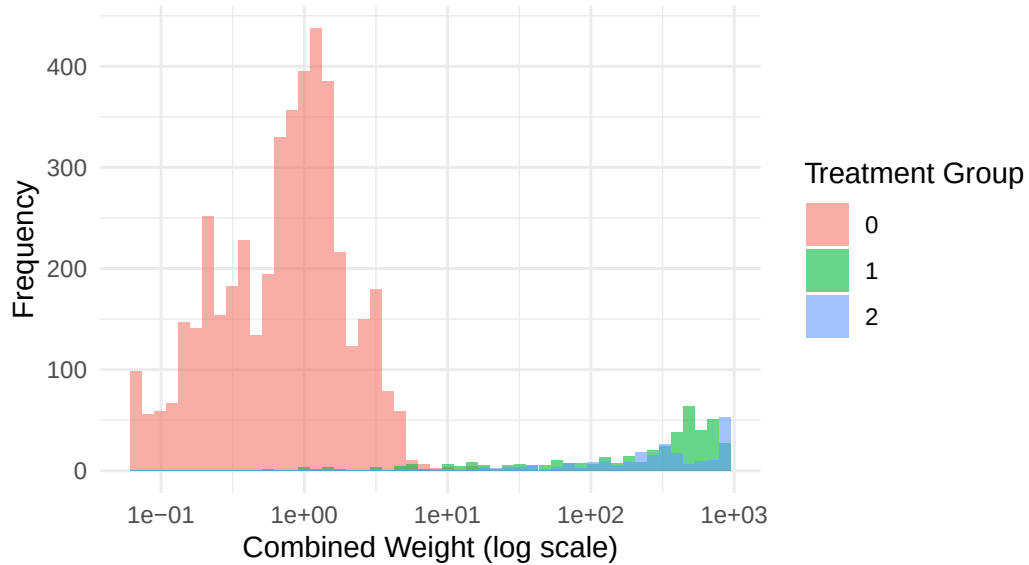
```
print(table(df_post$treatment_group))
```

```
  0   1   2
4450 404 215
```

```
# 重みの分布可視化
```

```
ggplot(df_post, aes(x = w_combined_trimmed, fill = treatment_group)) +
  geom_histogram(alpha = 0.6, bins = 50, position = "identity") +
  scale_x_log10() + labs( x = "Combined Weight (log scale)", y =
"Frequency", fill = "Treatment Group", title = "Distribution of Combined
IPTW × Sampling Weights" ) + theme_minimal()
```

Distribution of Combined
IPTW × Sampling Weights



群別アウトカムの可視化

```
outcome_plot <- ggplot(df_post, aes(x = factor(treatment_group), y =
nt_ch_stunt, weight = w_combined_trimmed)) + geom_boxplot(aes(fill =
factor(treatment_group)), alpha = 0.7) + labs(x = "Treatment Group
(0=Control, 1=Early, 2=Late)", y = "Stunting Outcome", fill = "Group",
title = "Weighted Distribution of Stunting by Treatment Group") +
theme_minimal()
```

```
print(outcome_plot)
```

